7CS039 Lab 6 Applications II

1 Introduction

Today we'll do something completely different. We are going to look at the `oneR` package written by Holger K. von Jouanne-Diedrich who runs the 'Learning Machines' blog at the link below

https://blog.ephorie.de/

Everything we do today is either **taken from Holger's blog** or from the oneR documentation.

2 Classification

The OneR algorithm is a classic algorithm for supervised learning. It was first described by Robert Holte in the nineties in his article

```
[1] Holte, Robert C. "Very simple classification rules perform well on
most commonly used datasets." Machine learning 11.1 (1993): 63-90.
```

It is a is a simple, yet accurate, classification algorithm that generates one rule for each predictor variable in the data. It then selects the rule with the smallest error as its "one rule". To create a rule for a predictor variable, we construct a frequency table for each predictor variable against the outcome we are trying to predict. OneR produces rules which are only slightly less accurate than some of the most sophisticated state-of-the-art classification algorithms while at the same time producing rules that are easy for us to interpret.

2.1 How it works?

In the following toy example let's suppose we are trying to come up with a rule that determines when our next door neighbour goes jogging. We suspect that it may have something to do with the weather so we collect data on fourteen days and obtain the data below.

Forecast	Temperature	Humidity	Wind	Go Jogging
Wet	High	High	No	No
Wet	High	High	Yes	No
Cloudy	High	High	No	Yes
Sun	Medium	High	No	Yes
Sun	Low	Normal	No	Yes
Sun	Low	Normal	Yes	No
Cloudy	Low	Normal	Yes	Yes
Wet	Medium	High	No	No
Wet	Low	Normal	No	Yes
Sun	Medium	Normal	No	Yes
Wet	Medium	Normal	Yes	Yes
Cloudy	Medium	High	Yes	Yes
Cloudy	High	Normal	No	Yes
Sun	Medium	High	Yes	No

We have four predictor variables; Forecast, Temperature, Humidity and Wind and we want to predict the target variable; Go Jogging. We make a frequency table for each predictor.

- Forecast

Go Jogging	Yes	No
Sun	3	2
Cloudy	4	0
Wet	2	3

  Now we pick the majority role for each feature i.e.

  - `if forecast = Sun then Go Jogging = Yes`
  - `if forecast = Cloudy then Go Jogging = Yes`
  - `if forecast = Wet then Go Jogging = No`

  These rules will correctly classify ten out of the fourteen rows in the dataset i.e. the accuracy is $\frac{10}{14} = 0.71$. They incorrectly classify four of the rows i.e. the error is $\frac{4}{14} = 0.29$.

- Temperature

Go Jogging	Yes	No
High	2	2
Medium	4	2
Low	3	1

  Now we pick the majority role for each feature i.e.

  - `if Temperature = High then Go Jogging = Yes` (or even no!)

- `if Temperature = Medium then Go Jogging = Yes`
- `if Temperature = Low then Go Jogging = Yes`

These rules will correctly classify nine out of the fourteen rows in the dataset i.e. the accuracy is $\frac{9}{14} = 0.64$. They incorrectly classify five of the rows i.e. the error is $\frac{5}{14} = 0.36$.

- Humidity

Go Jogging	Yes	No
High	3	4
Normal	6	1

Now we pick the majority role for each feature i.e.

- `if Humidity = High then Go Jogging = No`
- `if Humidity = Normal then Go Jogging = Yes`

These rules will correctly classify ten out of the fourteen rows in the dataset i.e. the accuracy is $\frac{10}{14} = 0.71$. They incorrectly classify four of the rows i.e. the error is $\frac{4}{14} = 0.29$.

- Wind

Go Jogging	Yes	No
No	6	2
Yes	3	3

Now we pick the majority role for each feature i.e.

- `if Wind = No then Go Jogging = Yes`
- `if Wind = Yes then Go Jogging = Yes` (or no!)

These rules will correctly classify nine out of the fourteen rows in the dataset i.e. the accuracy is $\frac{9}{14} = 0.64$. They incorrectly classify four of the rows i.e. the error is $\frac{5}{14} = 0.36$.

We see that we can acheive 71% accuracy wuth the following decision rules.

- `if forecast = Sun then Go Jogging = Yes`
- `if forecast = Cloudy then Go Jogging = Yes`
- `if forecast = Wet then Go Jogging = No`

Indeed we could also acheive this accuracy with

- `if Humidity = High then Go Jogging = No`
- `if Humidity = Normal then Go Jogging = Yes`

3 The OneR Package

The first thing we'll need to do is install the package and load it.

```
> library(OneR)
```

```
** testing if installed package can be loaded from final location
** testing if installed package keeps a record of temporary installation path
* DONE (OneR)

The downloaded source packages are in
        '/tmp/RtmpZJ7Bv0/downloaded_packages'
> library(OneR)
>
```

Data science & Artificial Intelligence are two closely linked fields and sometimes it's hard to tell if they are even different at all. But, no matter what course you are doing now you will need to know a little about machine learning and classification before you graduate.

Definition 3.1. Classification is the problem of identifying to which of a set of categories a new observation belongs, on the basis of a training set of data containing observations whose category membership is known.

4 Mushrooms

The traditional way coding a computer program worked was through carefully analyzing the problem, trying to separate its different parts into simpler sub-problems, put the solution into an algorithmic form and finally code it. Let's have a look at an example!

Let's say you want to program an app for mushroom pickers with warnings for certain qualities. The idea is to find an easy to follow guide in the form of "if your mushroom has this quality then DO NOT eat it!" (in computer lingo called "conditional statements" or just "conditionals"). This is not an easy task. We'll use the `mushrooms` data set which is on canvas but originally taken from the University of California, Irvine, Machine Learning Repository (a great resource for data scientists and AI professionals)

https://archive.ics.uci.edu/ml/datasets/mushroom

Load and preview the data as normal

```
> head(mushrooms)
# A tibble: 6 x 23
  cap_shape cap_surface cap_color bruises odor  gill_attachment gill_spacing gill_size
  <chr>     <chr>       <chr>     <chr>   <chr> <chr>           <chr>        <chr>
1 convex    smooth      brown     yes     pung… free            close        narrow
2 convex    smooth      yellow    yes     almo… free            close        broad
3 bell      smooth      white     yes     anise free            close        broad
4 convex    scaly       white     yes     pung… free            close        narrow
5 convex    smooth      gray      no      none  free            crowded      broad
6 convex    scaly       yellow    yes     almo… free            close        broad
# … with 15 more variables: gill_color <chr>, stalk_shape <chr>, stalk_root <chr>,
#   stalk_surface_above_ring <chr>, stalk_surface_below_ring <chr>,
#   stalk_color_above_ring <chr>, stalk_color_below_ring <chr>, veil_type <chr>,
#   veil_color <chr>, ring_number <chr>, ring_type <chr>, spore_print_color <chr>,
#   population <chr>, habitat <chr>, type <chr>
>
```

The dataset consists of 8124 examples of mushrooms with 22 qualities each (plus the attribute whether the respective mushroom is edible or poisonous). A naive approach would be to formulate rules for every instance: **if** the cap shape is convex **and** the cap surface smooth **and** the cap colour brow ...and so on for all 22 attributes, **then** DO NOT eat it! This would obviously not be very helpful. Another approach would be to go through every attribute and see whether it is helpful in determining the type, so for example:

```
> table(mushrooms$cap_shape, mushrooms$type)
```

```
> table(mushrooms$cap_shape, mushrooms$type)

         edible poisonous
  bell      404        48
  conical     0         4
  convex   1948      1708
  flat     1596      1556
  knobbed   228       600
  sunken     32         0
>
```

So, looking at the table we can see that e.g. if the mushroom is 'sunken' it's probably safe to eat, but then again there are only 32 sunken mushrooms. Whereas, if the mushroom is 'knobbed' then you probably don't want to eat. on the other hand if it's convex then you can't be sure at all. Obviously this attribute isn't very helpful, in many cases it just gives a 'maybe, maybe not' answer. Perhaps the approach itself is not so bad after all but you would have to try it for all 22 attributes, interpret the results, pick the best one, formulate if-then-rules and code them and this would be tiresome and error-prone.

Wouldn't it be nice to do it the other way around: just show the computer all of the examples and it magically programs itself by finding the appropriate if-then-rules

automatically. This is what AI is all about. So, let us throw AI at our problem in the form of the OneR package.

```
> mush_1R <- OneR(type ~., data = mushrooms, verbose=TRUE)
```

```
> mush_1R <- OneR(type ~., data = mushrooms, verbose=TRUE)

    Attribute                Accuracy
1 * odor                     98.52%
2   spore_print_color        86.8%
3   gill_color               80.5%
4   ring_type                77.55%
5   stalk_surface_above_ring 77.45%
6   stalk_surface_below_ring 76.61%
7   gill_size                75.63%
8   bruises                  74.4%
9   population               72.18%
10  stalk_color_above_ring   71.64%
11  stalk_color_below_ring   71.44%
12  habitat                  69.03%
13  stalk_root               64.6%
14  gill_spacing             61.6%
15  cap_color                59.53%
16  cap_surface              58.05%
17  cap_shape                56.43%
18  stalk_shape              55.29%
19  ring_number              53.82%
20  veil_color               51.9%
21  gill_attachment          51.8%
21  veil_type                51.8%
---
```

Now let's ask for more information

```
> summary(mush_1R)
```

```
Call:
OneR.formula(formula = type ~ ., data = mushrooms, verbose = TRUE)

Rules:
If odor = almond   then type = edible
If odor = anise    then type = edible
If odor = creosote then type = poisonous
If odor = fishy    then type = poisonous
If odor = foul     then type = poisonous
If odor = musty    then type = poisonous
If odor = none     then type = edible
If odor = pungent  then type = poisonous
If odor = spicy    then type = poisonous
```

Wow! Within the blink of an eye and with just one command (`OneR`) we got all the rules we need. It is the odour: if it smells poisonous it probably is. The accuracy of this is nearly 99%, not too bad for such a simple rule.
So, what is all of this about?

5 Introduction to oneR - adapted verbatim from the oneR Manual

The following story is one of the most often told in the Data Science community: some time ago the military built a system which aim it was to distinguish military vehicles from civilian ones. They chose a neural network approach and trained the system with pictures of tanks, humvees and missile launchers on the one hand and normal cars, pickups and trucks on the other. After having reached a satisfactory accuracy they brought the system into the field (quite literally). It failed completely, performing no better than a coin toss. What had happened? No one knew, so they re-engineered the black box (no small feat in itself) and found that most of the military pics where taken at dusk or dawn and most civilian pics under brighter weather conditions. The neural net had learned the difference between light and dark!

Although this might be an urban legend the fact that it is so often told tells us something:

1. Many of our Machine Learning models are so complex that we cannot understand them ourselves. Because of 1. we cannot differentiate between the simpler aspects of a problem which can be tackled by simple models and the more sophisticated ones which need specialized treatment.

2. The above is not only true for neural networks (and especially deep neural networks) but for most of the methods used today, especially Support Vector Machines and Random Forests and in general all kinds of ensemble based methods.

In one word: we need a good baseline which builds "the best simple mode" that strikes a balance between the best accuracy possible with a model that is still simple enough to

understand: I (Holger) have developed the OneR package for finding this sweet spot and thereby establishing a new baseline for classification models in Machine Learning (ML).

5.1 Design principles for the OneR package

The following design principles were followed for programming the package:

- Easy: the learning curve for new users should be minimal. Results should be obtained with ease and only minimal preprocessing and modeling steps should be necessary.
- Versatile: all types of data, i.e. categorical and numeric, should be computable - as input variable as well as as target.
- Fast: the running times of model trainings should be short.
- Accurate: the accuracy of trained models should be good overall.
- Robust: models should not be prone to overfitting; the reached accuracy on training data should be comparable to the accuracy of predictions from new, unseen cases.
- Comprehensible: it should be easy to understand which rules the model has learned. Not only should the rules be easily comprehensible but they should serve as heuristics that are usable even without a computer.
- Reproducible: because the used algorithms are strictly deterministic one will always get the same models on the same data. Many ML algorithms have stochastic components so that the data scientist will get a different model every time.
- Intuitive: model diagnostics should be presented in form of simple tables and plots.
- Native R: the whole package is written in native R code. Thereby the source code can be easily checked and the whole package is very lean. Additionally the package has no dependencies at all other than base R itself.

The package is based on the one rule classification algorithm. Although the underlying method is simple enough several enhancements have been made:

- Discretization of numeric data: the OneR algorithm can only handle categorical data, so numeric data has to be discretized. The original OneR algorithm separates the respective values in ever smaller and smaller buckets until the best possible accuracy is being reached. It can be argued that this is the definition of overfitting and contradicts the original spirit of OneR because tons of rules (one for every bucket) will result. One can of course introduce a new parameter "maximum bucket size" but finding the right value for this one doesn't come naturally either. Therefore I take a radically different approach: there are several methods for handling numeric data in the package (in the bin and the optbin function), the most promising one is the (default) "logreg" method in the optbin function which gives only as many bins as there are target categories and which optimizes the cut points according to pairwise logistic regressions.

- Missing values: in the original algorithm missing values were always handled as a separate level in the respective attribute. While missing values can sometimes reveal interesting patterns in other cases they are, well, just values that are missing. In the OneR package missing values can be handled as separate levels (level "NA") or they can be omitted (the default).

- Tie breaking: sometimes the OneR algorithm will find several attributes that provide rules which all give the same best accuracy. The original algorithm just took the first attribute. While this is implemented in the OneR function as the default too a different method for tie breaking can be chosen: the contingency tables of all "best" rules are tested against each other with a Pearson's Chi squared test and the one with the smallest p-value is being chosen. The rationale behind this is that thereby the attribute with the best signal-to-noise ratio is being found.

6 Iris Example

We're going to use the famous Iris data set.

```
> head(iris)
```

```
> head(iris)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
>
```

The first thing we need to do determine optimal bins for the numerical data.

```
> data <- optbin(iris)
```

```
> data <- optbin(iris)
> head(data)
  Sepal.Length Sepal.Width Petal.Length    Petal.Width Species
1   (4.3,5.41]  (3.19,4.4] (0.994,2.46] (0.0976,0.791]  setosa
2   (4.3,5.41] (2.87,3.19] (0.994,2.46] (0.0976,0.791]  setosa
3   (4.3,5.41]  (3.19,4.4] (0.994,2.46] (0.0976,0.791]  setosa
4   (4.3,5.41] (2.87,3.19] (0.994,2.46] (0.0976,0.791]  setosa
5   (4.3,5.41]  (3.19,4.4] (0.994,2.46] (0.0976,0.791]  setosa
6   (4.3,5.41]  (3.19,4.4] (0.994,2.46] (0.0976,0.791]  setosa
>
```

Now we build our predictive model

```
> model <- OneR(data, verbose = TRUE)
```

```
> model <- OneR(data, verbose = TRUE)

    Attribute    Accuracy
1 * Petal.Width  96%
2   Petal.Length 95.33%
3   Sepal.Length 74.67%
4   Sepal.Width  55.33%
---
Chosen attribute due to accuracy
and ties method (if applicable): '*'

>
```

We see that the strongest attribute to predict the type of flower is petal width (closely followed by petal length). We can ask for a summary of the model

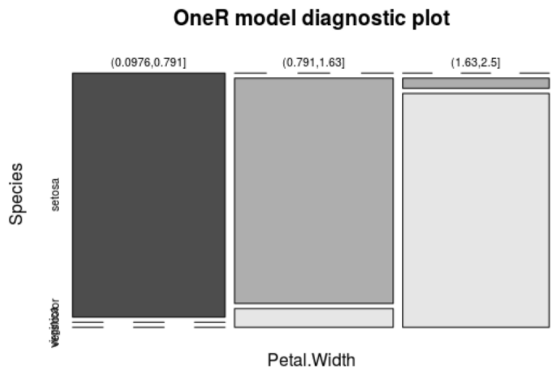
```
> summary(model)
```

```
> summary(model)

Call:
OneR.data.frame(x = data, verbose = TRUE)

Rules:
If Petal.Width = (0.0976,0.791] then Species = setosa
If Petal.Width = (0.791,1.63]   then Species = versicolor
If Petal.Width = (1.63,2.5]     then Species = virginica

Accuracy:
144 of 150 instances classified correctly (96%)

Contingency table:
            Petal.Width
Species      (0.0976,0.791] (0.791,1.63] (1.63,2.5] Sum
  setosa               * 50           0          0  50
  versicolor              0         * 48          2  50
  virginica               0            4       * 46  50
  Sum                    50           52         48 150
---
Maximum in each column: '*'

Pearson's Chi-squared test:
X-squared = 266.35, df = 4, p-value < 2.2e-16

>
```

There is also a built in plotting function for `oneR` models.

```
> plot(model)
```

```
OneR.data.frame(x = data, verbose = TRUE)

Rules:
If Petal.Width = (0.0976,0.791] then Species = setosa
If Petal.Width = (0.791,1.63]   then Species = versicolor
If Petal.Width = (1.63,2.5]     then Species = virginica

Accuracy:
144 of 150 instances classified correctly (96%)

Contingency table:
            Petal.Width
Species      (0.0976,0.791] (0.791,1.63] (1.63,2.5] Sum
  setosa               * 50           0          0  50
  versicolor              0        * 48          2  50
  virginica               0           4       * 46  50
  Sum                    50          52         48 150
---
Maximum in each column: '*'

Pearson's Chi-squared test:
X-squared = 266.35, df = 4, p-value < 2.2e-16

> plot(model)
>
```



This is a mosaic plot and what we are looking for us that the three areas are separate from, each other in the vertical bars. The nearly perfect separation of the areas in the diagnostic plot gives a good indication of the model's ability to separate the different species. We can evaluate the model's predictive quality.

```
> prediction <- predict(model, data)
> eval_model(prediction, data)
```

```
> prediction <- predict(model, data)
> eval_model(prediction, data)

Confusion matrix (absolute):
            Actual
Prediction   setosa versicolor virginica Sum
  setosa         50          0         0  50
  versicolor      0         48         4  52
  virginica       0          2        46  48
  Sum            50         50        50 150

Confusion matrix (relative):
            Actual
Prediction   setosa versicolor virginica  Sum
  setosa       0.33       0.00      0.00 0.33
  versicolor   0.00       0.32      0.03 0.35
  virginica    0.00       0.01      0.31 0.32
  Sum          0.33       0.33      0.33 1.00

Accuracy:
0.96 (144/150)

Error rate:
0.04 (6/150)

Error rate reduction (vs. base rate):
0.94 (p-value < 2.2e-16)

>
```

A confusion matrix is an $N \times N$ matrix used for evaluating the performance of a classification model, where N is the number of target classes. The matrix compares the actual

target values with those predicted by the machine learning model. This gives us a holistic view of how well our classification model is performing and what kinds of errors it is making. In our dataset we have fifty of each type of flower so we can see in the output where our model makes mistakes.

7 Exercise

Visit

https://cran.r-project.org/web/packages/OneR/vignettes/OneR.html

and work through example based on the `breastcancer` dataset.